

LLM Routing: From Query-Level Model Selection to Agentic Multi-Round Routing

A Comparative Literature Review

by

Shulun Zhang

225045020

Final Report

Prepared for the Operating Systems course
in partial fulfillment of the course requirements

School of Data Science
The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Large language model (LLM) routing has emerged as a central layer in practical AI deployment because contemporary systems increasingly rely on heterogeneous model pools rather than a single universal model. Different models offer different trade-offs in quality, cost, latency, and domain specialization, so invoking the same model for every request is often inefficient. This report studies three representative papers that illustrate the recent evolution of routing methods: RouteLLM, GraphRouter, and Router-R1. Respectively, these papers represent preference-based binary routing, contextual multi-model routing, and agentic multi-round routing.

The main argument of this report is that recent LLM routing research has evolved along three connected axes: routing granularity, context representation, and decision adaptivity. RouteLLM makes lightweight one-shot escalation decisions from preference supervision. GraphRouter expands the routing state by modeling relations among tasks, queries, and models in a heterogeneous graph. Router-R1 further extends the decision space by allowing routing to unfold over multiple rounds through reinforcement-learned action sequences. Based on a unified comparative framework, this report analyzes each paper’s problem setting, core method, evaluation design, strengths, limitations, and position in the broader routing literature. It concludes that the field is moving toward richer and more adaptive routing policies, while still facing open challenges in evaluation consistency, online adaptation, latency-aware control, and integration with real deployment constraints.

Contents

Abstract	i
1 Introduction	1
1.1 Research Questions	2
1.2 Selection of the Three Focal Papers	3
1.3 Contributions of This Review	3
1.4 Methodological Posture	4
1.5 Chapter Outline	4
2 Background and Preliminaries	5
2.1 Why LLM Routing Matters	5
2.2 Analytical Framework for LLM Routers	5
2.3 Formal Problem Statement	6
2.4 Broader Related Work on LLM Routing	7
2.5 Benchmarks and Evaluation Infrastructure	7
2.6 Benchmark Source Papers and What They Measure	10
2.7 Evaluation Metrics and Their Meanings	11
2.8 Scope and Limitations of This Review	12
2.9 Threats to Cross-Paper Comparison	13
2.10 Summary	13
3 RouteLLM: Preference-Based Binary Routing	14
3.1 Motivation and Problem Setting	14
3.2 Core Formulation	14
3.3 Supervision and Data Construction	15
3.4 Router Families	15
3.5 Architectural Instantiations in More Detail	16
3.6 Benchmarks and Evaluation	17
3.7 Experimental Findings Across Benchmarks	17
3.8 Adaptation Across Model Pairs	20
3.9 Cost Savings, Overhead, and Deployment Interpretation	20

3.10	Strengths	20
3.11	Limitations	22
3.12	Position in the Evolution of LLM Routing	22
3.13	Summary	22
4	GraphRouter: Graph-Based Multi-Model Selection	23
4.1	Problem Setting and Motivation	23
4.2	Graph Construction	23
4.3	Inductive Initialization and Heterogeneous GNN	24
4.4	Datasets and Experimental Setting	24
4.5	Baselines and Evaluation Scenarios	25
4.6	Results and Interpretation	26
4.7	Generalization to Newly Introduced LLMs	26
4.8	Ablations and Extended Experiments	28
4.9	Strengths	29
4.10	Limitations	29
4.11	Position in the Evolution of LLM Routing	30
4.12	Summary	30
5	Router-R1: Agentic Multi-Round Routing	31
5.1	Why One-Shot Routing Is Sometimes Not Enough	31
5.2	Sequential RL Formulation	31
5.3	Reward Curation	32
5.4	Action Protocol and Prompt Structure	32
5.5	Training Prompt and Interaction Loop	33
5.6	Benchmarks and Results	34
5.7	Baselines and Experimental Protocol	34
5.8	Main Results in Comparative Perspective	35
5.9	Cost-Aware Routing and Pareto Trade-offs	37
5.10	Generalization to Unseen Candidate Models	38
5.11	Behavioral Analysis and Descriptor Sensitivity	38
5.12	Strengths	39
5.13	Limitations and Costs	39
5.14	Relation to the Broader RL and Reasoning Literature	39
5.15	Position in the Evolution of LLM Routing	40
5.16	Cross-Paper Comparison	40
5.17	Practical Design Trade-offs	41
5.18	Summary	41

6	Cross-Paper Synthesis and Open Challenges	42
6.1	Revisiting the Three Evolutionary Axes	42
6.2	Supervision, Data Construction, and Labeling Burden	42
6.3	Evaluation Regimes and Their Hidden Assumptions	43
6.4	Deployment Suitability and Workload Matching	43
6.5	Open Problems	44
6.6	Summary	44
7	Conclusion and Future Work	46
A	Supplementary Notes on Benchmarks and Model Pools	47
A.1	Additional Notes on Benchmark Source Papers	47
A.2	Supplementary Notes on Candidate Model Pools	49
A.3	Closing Note on Supplementary Material	50

List of Figures

1.1	Evolution of LLM routing paradigms. The figure is redrawn by the author to summarize the literature reviewed in this report.	2
3.1	RouteLLM evaluates routers by the relationship between strong-model call rate and recovered performance. The figure is redrawn by the author based on RouteLLM [1].	18
3.2	Selected APGR values from RouteLLM show that benchmark-aligned data augmentation substantially improves routing quality. The chart is redrawn by the author based on results reported in RouteLLM [1].	18
4.1	GraphRouter transforms interaction records into a heterogeneous graph and performs typed aggregation before edge prediction. The figure is redrawn by the author based on GraphRouter [2].	25
4.2	GraphRouter’s advantage persists across performance-first, balanced, and cost-first scenarios. The chart is redrawn by the author based on results reported in GraphRouter [2].	28
5.1	Router-R1 turns routing into a multi-round procedure in which the policy alternates between reasoning and external model invocation. The figure is redrawn by the author based on Router-R1 [3].	33
5.2	Router-R1 explicitly studies the impact of the cost coefficient α on answer quality and cost reward. The figure is redrawn by the author based on Router-R1 [3].	34
5.3	Router-R1 improves average exact-match over direct inference, Search-R1, and GraphRouter on both evaluated backbones. The chart is redrawn by the author based on results reported in Router-R1 [3].	36

List of Tables

2.1	Representative families in the broader LLM routing literature.	8
2.2	Representative stages in the evolution of LLM routing.	8
2.3	Major datasets and benchmarks mentioned in the three focal papers. . .	9
2.4	Benchmark families and what they contribute to routing evaluation. . . .	11
2.5	Evaluation focus across the three focal papers.	12
3.1	RouteLLM router families and their design trade-offs.	16
3.2	Selected RouteLLM results reported across the three main benchmarks. .	19
3.3	RouteLLM deployment-oriented observations reported in the paper. . . .	21
4.1	Compact view of GraphRouter’s experimental setting.	26
4.2	Selected GraphRouter results under different deployment scenarios. . . .	27
4.3	GraphRouter in the few-shot new-LLM setting.	29
5.1	Compact view of Router-R1’s training and evaluation protocol.	35
5.2	Selected average exact-match results reported for Router-R1.	36
5.3	Illustrative cost–accuracy trade-off for Router-R1 on Natural Questions. .	37
5.4	Average exact-match before and after adding unseen candidate LLMs to the routing pool.	38
5.5	Comparison of the three focal papers.	40
6.1	Open problems suggested by the three focal papers.	44
A.1	Supplementary notes on benchmark source papers and their routing rele- vance.	48
A.2	Supplementary comparison of model-pool assumptions across the three focal papers.	50

Chapter 1

Introduction

Large language models are no longer deployed as isolated monolithic systems. In practice, modern AI services increasingly expose a pool of heterogeneous models with different capabilities and costs. A powerful proprietary model may provide the best answer quality, but its latency and API price can be prohibitive. A smaller open model may be fast and inexpensive, but its reasoning ability may be insufficient for difficult queries. Domain-specific models can outperform larger general-purpose models on certain tasks while remaining worse on average. Under these conditions, invoking a single model for every request is usually an inefficient policy. The more realistic question is which model should be used for a particular query under a given quality target, latency budget, and cost constraint.

This question motivates the study of LLM routing. In its simplest form, an LLM router receives a user query and chooses one model from a candidate pool. In richer settings, the router may use contextual signals such as task identity, interaction history, model descriptors, or benchmark-derived performance priors. In the most advanced settings, the router may make several decisions during a single query, interleaving reasoning, external model calls, and evidence integration. Recent work suggests that routing is no longer a narrow implementation detail. It is becoming a distinct research area concerned with how to allocate heterogeneous model resources under changing quality-cost trade-offs.

Recent routing methods also reveal a clear methodological evolution. Early work largely focused on binary strong-versus-weak switching. More recent approaches consider many models and explicitly represent relations among tasks, queries, and model characteristics. The latest work goes further by treating routing as a sequential control problem in which decisions can be revised after intermediate reasoning or external information gathering. This progression can be summarized along three analytical axes that structure the present report: routing granularity, context representation, and decision adaptivity.

This report studies three focal papers chosen because they represent different points

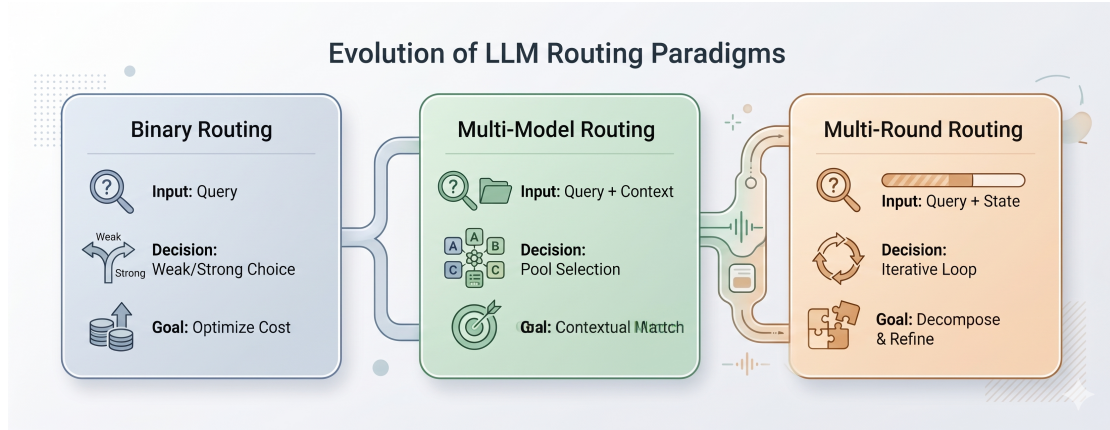


Figure 1.1: Evolution of LLM routing paradigms. The figure is redrawn by the author to summarize the literature reviewed in this report.

in that evolution. RouteLLM [1] is a representative preference-based binary router for premium-model escalation. GraphRouter [2] is a representative contextual multi-model router built around relational representation learning. Router-R1 [3] is a representative agentic multi-round router in which a policy LLM plans, calls external models, and updates its decisions over time. The report asks how these three methods differ in the information they use, the decisions they make, the supervision they require, and the deployment trade-offs they create.

The contribution of this report is therefore not only to summarize three papers, but also to compare them under a unified framework. The analysis focuses on five dimensions: decision space, context representation, supervision and optimization, generalization target, and routing overhead.

1.1 Research Questions

To make the comparative logic explicit, this review is organized around three research questions. The first question is how the *granularity* of routing has changed. Binary routers such as RouteLLM choose between a weak and a strong model; contextual routers such as GraphRouter choose from a larger pool; sequential routers such as Router-R1 make multiple decisions over time. The second question is how the *representation of routing context* has evolved. Early routers rely primarily on query text, while later methods incorporate task labels, interaction graphs, model descriptors, and intermediate reasoning states. The third question is how much *adaptivity* is worth its cost. Richer routing policies promise better task fit and better use of complementary model strengths, but they also require more supervision, more runtime overhead, and more careful evaluation.

These questions are especially important because recent routing papers are not simply variants of the same algorithm. They redefine what counts as a routing state, what

supervision signal is legitimate, and what deployment target is being optimized. A thesis-style review should therefore compare more than benchmark numbers. It should compare the assumptions each paper makes about workloads, model pools, labeling resources, and runtime constraints.

1.2 Selection of the Three Focal Papers

The three focal papers were selected because they mark distinct points in the recent evolution of LLM routing. RouteLLM [1] is representative of the binary escalation setting that arises when a deployment contains one expensive high-quality model and one cheaper fallback model. It is also representative in methodological terms because it replaces heuristic dispatch rules with preference-supervised learning from human comparisons. GraphRouter [2] represents the next expansion of the design space: routing is no longer framed as strong-versus-weak classification, but as relational selection over a broader ecosystem of tasks, queries, and candidate models. Router-R1 [3] then pushes the boundary further by turning routing into a sequential policy problem with explicit reasoning, repeated model invocation, and aggregation.

Taken together, these papers provide a useful comparative arc. They do not cover every possible routing method, but they are sufficient to expose the main transitions in the literature: from binary to multi-model decision spaces, from query-only to structured context, and from one-shot dispatch to multi-round control. They also use sufficiently different supervision and evaluation regimes that a careful comparison yields broader lessons about how routing research is being framed.

1.3 Contributions of This Review

This report makes four concrete contributions as a comparative literature review. First, it reorganizes three focal papers under a common analytical framework rather than presenting them as isolated summaries. Second, it reconnects the methods to the benchmark and dataset source papers they rely on, which is important because routing claims are inseparable from the distributions on which they are evaluated. Third, it expands the comparison to include design trade-offs in supervision, model-pool assumptions, generalization targets, and deployment overhead. Fourth, it situates the three focal papers in the broader routing literature, including FrugalGPT [4], HybridLLM [5], RouterDC [6], C2MAB-V [7], Fly-Swat [8], TensorOpera Router [9], and RouterBench [10].

1.4 Methodological Posture

The methodological posture of this report is interpretive rather than purely empirical. I do not independently re-implement the three focal systems. Instead, I read the primary papers, inspect their formalizations, compare their benchmarks and reported results, and evaluate the coherence of their claims under a shared set of comparative dimensions. This means that the strengths of the report lie in structured synthesis, critical comparison, and literature positioning. Its limitations are equally important: all quantitative claims ultimately depend on the experimental settings reported in the original papers, and direct cross-paper numerical comparisons must be treated carefully whenever the datasets, candidate model pools, or metrics differ.

1.5 Chapter Outline

The rest of the report is organized as follows. Chapter 2 presents the background, broader related work, benchmark context, and analytical framework used throughout the review. Chapters 3 to 5 discuss RouteLLM, GraphRouter, and Router-R1 in a parallel structure, with each chapter covering problem setting, method, supervision, experiments, strengths, limitations, and evolutionary position. Chapter 6 synthesizes the comparison across the three papers and develops a broader discussion of open problems in LLM routing. Chapter 7 concludes.

Chapter 2

Background and Preliminaries

2.1 Why LLM Routing Matters

The demand for LLM routing arises directly from the heterogeneity of modern inference services. Model providers now offer families of models rather than one flagship system. These models differ in size, context window, reasoning ability, domain specialization, latency, and per-token price. Open models further diversify the landscape by allowing systems to trade monetary cost for engineering overhead and deployment complexity. In addition, evaluation results are highly task-dependent: a model that performs strongly on conversational benchmarks may underperform on mathematical reasoning, code generation, or multi-hop question answering.

This heterogeneity creates a deployment dilemma. Always calling the strongest model maximizes average quality but wastes cost on easy queries. Always calling the cheapest model minimizes cost but fails on hard queries. Therefore, the objective of routing is not simply to find the “best” model in the abstract. Instead, it is to learn an allocation policy that matches each query to an appropriate model or sequence of models. Prior work such as FrugalGPT [4], HybridLLM [5], C2MAB-V [7], RouterDC [6], TensorOpera Router [9], and RouterBench [10] demonstrates that this problem has become important enough to support both standalone methods and benchmarking infrastructures.

2.2 Analytical Framework for LLM Routers

To compare routing methods in a principled way, this report uses a unified analytical framework. Let q denote an incoming query and $\mathcal{M}$ a set of candidate models. A router seeks a policy π that maps q to either a single model $M \in \mathcal{M}$ or to a sequence of routing actions. Although this abstract formulation is simple, the design space expands quickly once one asks what information the router can use, how many decisions it can make, and what objective it is trained to optimize.

The first dimension is *routing granularity*. Some methods make a binary decision between a weak and a strong model. Others choose among many models in one shot. The most adaptive methods make several decisions during one query and may revise earlier choices. The second dimension is *context representation*. Routers differ in whether they rely only on query text, on richer relational context such as task and model metadata, or on trajectory-aware state that includes intermediate observations. The third dimension is *supervision and optimization*. Existing methods range from preference supervision, to logged interaction learning, to reinforcement learning with explicit outcome and cost rewards.

Two further dimensions are especially important for comparative review. One is the *generalization target*: some routers primarily generalize across strong-weak pairs, others across unseen models or tasks, and others across changing reasoning trajectories and model pools. The other is *routing overhead*, including data requirements, inference latency, and control complexity. These dimensions help explain why more expressive routers are not automatically better for every workload.

2.3 Formal Problem Statement

It is useful to make the common structure across these papers explicit. In the single-round setting, a router receives a query q and chooses a model $m \in \mathcal{M}$ from a candidate pool:

$$\pi_{\text{single}}(q) \rightarrow m. \quad (2.1)$$

The quality of this decision depends on the expected utility of using model m for q . A generic objective can be written as

$$U(q, m) = Q(q, m) - \lambda C(q, m) - \mu L(q, m), \quad (2.2)$$

where Q denotes expected answer quality, C denotes monetary or compute cost, and L denotes latency or delay. Different routing papers instantiate this objective differently. RouteLLM emphasizes the quality recovered by escalating to a stronger model. GraphRouter combines performance and cost in a weighted reward. Router-R1 folds answer correctness, output format, and cost into a sequential reward function.

In the sequential setting, the router observes not only the original query but also an evolving state s_t that may include prior actions, external model responses, and intermediate reasoning traces:

$$\pi_{\text{seq}}(s_t) \rightarrow a_t. \quad (2.3)$$

This formulation makes clear why Router-R1 belongs to a different regime from RouteLLM and GraphRouter. The action is no longer limited to choosing one model once. Instead,

the policy may think, call an external model, integrate the returned information, and continue. As a result, routing is no longer only about placement. It becomes a control problem over an evolving trajectory.

2.4 Broader Related Work on LLM Routing

The three focal papers should also be situated within the broader routing literature. Early work emphasized *cost-aware switching*. FrugalGPT [4] popularized the idea that inference can be organized as a cascade in which cheaper models handle easy cases and stronger models are reserved for difficult ones. HybridLLM [5] similarly studies dynamic routing between small and large models while explicitly considering user-specified quality budgets. These works establish the deployment motivation that makes RouteLLM compelling.

Another branch of the literature focuses on *learned query-based routing*. RouterDC [6] uses dual contrastive learning to align query and LLM representations. Fly-Swat [8] studies meta-modeling for cost-effective model choice, making explicit the fact that the router itself is a predictor over expected model utility. TensorOpera Router [9] explores lightweight router families and deployment-oriented trade-offs, while RouterBench [10] shows that evaluation itself has become a major concern as multi-LLM routing systems proliferate.

The broader literature also highlights two additional trends that matter for this report. First, there is a shift from static model identifiers to richer model descriptors, which makes inductive generalization more plausible in fast-changing model ecosystems. Second, there is a shift from single-shot selection to richer *reasoning-aware control*. Methods such as chain-of-thought prompting [11], Tree-of-Thoughts [12], Search-R1 [13], and DeepSeek-R1 [14] do not all solve routing in the same way, but they collectively normalize the idea that LLM inference can be decomposed into multi-step decision processes rather than treated as one monolithic answer generation event.

2.5 Benchmarks and Evaluation Infrastructure

The papers studied in this report rely on a broad set of benchmarks. Since benchmark choice strongly influences the meaning of reported performance, it is important to document them explicitly. Table 2.3 summarizes the most important datasets used by the three papers.

Table 2.1: Representative families in the broader LLM routing literature.

Literature family	Representative papers	Typical decision regime	Main limitation relative to later work
Cost-aware switching	FrugalGPT, HybridLLM	Strong-vs.-weak or small-vs.-large switching	Usually limited context representation and modest generalization to new model pools
Learned query routers	RouteLLM, RouterDC, Fly-Swat	One-shot selection from query features	Often rely on static model assumptions or limited relational context
Contextual multi-model routing	GraphRouter, C2MAB-V, TensorOpera Router	Choose among many models under explicit trade-offs	More expressive, but still usually one-shot and dependent on richer logs
Reasoning-aware routing	Router-R1, Search-R1-style systems	Multi-round action selection with intermediate state	Higher training, inference, and evaluation complexity
Benchmarking and evaluation infrastructure	RouterBench and related work	Compare policies under shared model pools and metrics	Benchmarks still struggle to match rapidly changing deployment conditions

Table 2.2: Representative stages in the evolution of LLM routing.

Stage	Representative methods	Main decision space	Main research shift
Cost-aware switching	FrugalGPT, HybridLLM, early binary routers	Choose whether to escalate to a larger model	Move beyond always-on premium inference toward selective model invocation
Preference-based routing	RouteLLM	Predict whether the strong model is worth calling	Learn escalation from preference supervision rather than fixed heuristics
Contextual multi-model routing	GraphRouter, RouterDC, C2MAB-V	Choose among many candidate models under explicit trade-offs	Enrich routing with task, query, and model relations instead of query-only signals
Agentic sequential routing	Router-R1, Search-R1-style systems	Interleave reasoning, external calls, and answer synthesis	Expand routing from one-shot selection to multi-round adaptive control

Table 2.3: Major datasets and benchmarks mentioned in the three focal papers.

Dataset / Benchmark	Source paper	Task type	Role in the routing literature
Chatbot Arena	[15]	Human preference comparison	Preference data source for RouteLLM; provides pairwise judgments between models
MMLU	[16]	Broad multitask knowledge	Evaluates general academic knowledge and out-of-domain generalization
MT-Bench	[17]	Open-ended dialogue evaluation	Measures conversational quality with LLM-as-a-judge
GSM8K	[18]	Grade-school mathematical reasoning	Tests structured reasoning and arithmetic competence
Alpaca	[19]	Instruction- following hybrid QA	Used by GraphRouter as part of a multi-task interaction dataset
SQuAD	[20]	Reading comprehension	Evaluates extraction and passage-grounded answer selection
Multi-News	[21]	Multi-document summarization	Tests summarization quality under different model choices
Natural Questions	[22]	Open-domain QA	In-domain training and evaluation source for Router-R1
TriviaQA	[23]	Distantly supervised QA	Out-of-domain evaluation for general QA routing
PopQA	[24]	Entity-centric factual QA	Tests robustness on knowledge-intensive factual questions
HotpotQA	[25]	Multi-hop QA	In-domain Router-R1 training and evaluation benchmark
2WikiMultiHop QA	[26]	Multi-hop QA	Evaluates reasoning across linked evidence chains
MuSiQue	[27]	Composed multi-hop QA	Measures more compositional and challenging multi-step reasoning

Dataset / Benchmark	Source paper	Task type	Role in the routing literature
Bamboogle	[28]	Compositional QA	Used to stress compositionality and reasoning beyond memorization

These datasets are not interchangeable. Chatbot Arena provides human preference comparisons rather than correctness labels. MMLU captures broad knowledge but not necessarily multi-step planning. GSM8K emphasizes mathematical reasoning, while MT-Bench emphasizes open-ended conversational quality. The Router-R1 benchmark suite further separates general QA from multi-hop QA, which is important because multi-round routing is most plausible when reasoning requires staged evidence gathering. Benchmark choice therefore shapes what “good routing” means in each paper, and direct numerical comparisons across papers should be interpreted with caution.

2.6 Benchmark Source Papers and What They Measure

The benchmark source papers themselves reveal important differences in evaluation semantics. Chatbot Arena [15] is based on pairwise human preferences over conversational outputs. It is therefore especially suitable for RouteLLM, whose supervision signal is explicitly comparative. MT-Bench [17] sits nearby in spirit because it evaluates open-ended assistant behavior with an LLM judge rather than relying on one exact gold answer. MMLU [16] and GSM8K [18] differ from these open-ended benchmarks because they emphasize correctness against structured academic or mathematical tasks.

GraphRouter’s benchmark mixture is distinctive. Alpaca [19] captures instruction-following behavior derived from self-instruct style data generation. SQuAD [20] emphasizes passage-grounded extraction and reading comprehension. Multi-News [21] tests multi-document summarization, which introduces a qualitatively different generation pattern from short-form QA. Because GraphRouter studies multiple tasks and multiple models at once, its evaluation claims are fundamentally about heterogeneity rather than about one benchmark family.

Router-R1 relies on a general-QA and multi-hop-QA suite whose source papers are also quite heterogeneous. Natural Questions [22] is grounded in real search queries and Wikipedia evidence. TriviaQA [23] is distant-supervised and information-rich. PopQA [24] probes factual knowledge and when parametric memory should or should not be trusted. HotpotQA [25], 2WikiMultiHopQA [26], MuSiQue [27], and Bamboogle [28] progressively

foreground compositional reasoning, evidence chaining, and multi-step dependency. This makes the Router-R1 evaluation suite especially suitable for studying whether multi-round routing is truly warranted.

Table 2.4: Benchmark families and what they contribute to routing evaluation.

Benchmark family	Representative datasets	What the family reveals about routing
Human preference and open-ended dialogue	Chatbot Arena, MT-Bench	Whether routing can preserve judged answer quality on conversational or open-ended tasks without always calling the strongest model
Knowledge and academic QA	MMLU, Natural Questions, TriviaQA, PopQA	Whether routers can discriminate between easy and difficult factual or academic questions under cost constraints
Structured reasoning and mathematics	GSM8K	Whether routing can detect when stronger reasoning models are needed rather than relying on cheap default inference
Reading comprehension and summarization	SQuAD, Multi-News	Whether routing generalizes across task forms beyond conversational QA, especially when outputs differ in length and evidence structure
Multi-hop and compositional QA	HotpotQA, 2WikiMultiHopQA, MuSiQue, Bamboogle	Whether one-shot routing is sufficient, or whether tasks with staged evidence dependencies benefit from multi-round coordination

2.7 Evaluation Metrics and Their Meanings

The three papers use different evaluation metrics because they target different routing regimes. RouteLLM focuses on cost-quality trade-offs between a strong and weak model. It therefore defines metrics such as performance gap recovered (PGR), average performance gap recovered (APGR), and call-performance threshold (CPT). GraphRouter evaluates performance, cost, and a weighted reward that trades the two off under different deployment scenarios. Router-R1 uses exact match and F1 over QA benchmarks while also studying cost-sensitive reward shaping through a tunable coefficient.

These choices are meaningful because routing quality is not identical to answer quality. A trivial policy that always calls the most expensive model may obtain the highest possible performance while failing the basic purpose of routing. Conversely, a policy that always chooses the smallest model may be cheap but inadequate. The correct evaluation target must therefore reflect the relationship between resource usage and answer quality.

This insight helps explain why routing papers often introduce their own metrics rather than relying only on standard ML evaluation. However, it also complicates comparative review. Some metrics evaluate deployment efficiency, some emphasize correctness, and some combine both in paper-specific ways. The present report therefore treats metric design itself as part of each paper’s contribution and limitation.

Table 2.5: Evaluation focus across the three focal papers.

Paper	Primary metrics	Benchmark style	What the evaluation is really testing
RouteLLM	PGR, APGR, CPT	Strong-vs.-weak routing on broad knowledge, dialogue, and math tasks	Whether a lightweight router can recover strong-model quality with fewer expensive calls
GraphRouter	Performance, cost, weighted reward	Multi-task, multi-model recommendation with explicit performance-cost scenarios	Whether contextual relational modeling improves one-shot placement across many candidate models
Router-R1	Exact Match, F1, cost-aware reward analysis	General QA plus multi-hop QA under multi-round routing	Whether sequential coordination actually helps on tasks that require staged evidence collection

2.8 Scope and Limitations of This Review

This report focuses on three representative papers rather than attempting a comprehensive survey of all LLM routing methods. The selection is intentional: RouteLLM, GraphRouter, and Router-R1 together span binary, contextual multi-model, and multi-round routing. At the same time, the report emphasizes routing design and evaluation logic rather than full serving-stack implementation details. It relies primarily on the original papers, their reported experiments, and the benchmark references they cite, rather than on independent re-implementation. As a result, the value of this review lies in structured comparison and critical synthesis rather than in reproducing empirical results.

2.9 Threats to Cross-Paper Comparison

There are several reasons why direct numerical comparison across the three focal papers is inherently limited. First, they do not share the same candidate model pools. RouteLLM studies a binary strong-weak setup, GraphRouter studies a richer multi-model ecosystem, and Router-R1 studies a sequential coordination problem with a policy LLM plus a routing pool. Second, they do not share the same notion of success. RouteLLM emphasizes recovered quality under reduced strong-model usage, GraphRouter emphasizes weighted reward under multiple scenarios, and Router-R1 emphasizes answer correctness plus cost-aware behavior on multi-hop tasks.

Third, the benchmark distributions are different enough that success on one paper’s evaluation suite does not imply success on another’s. For example, MT-Bench and Chatbot Arena emphasize open-ended judged quality, whereas HotpotQA and MuSiQue emphasize compositional evidence chaining. Fourth, prices, latencies, and available model families change rapidly in real deployment environments. A routing paper can therefore age quickly even if its conceptual contribution remains important. These threats do not invalidate comparative review, but they require the review to focus on design logic and experimental regime rather than on headline numbers alone.

2.10 Summary

This chapter has established the conceptual ground needed for the rest of the report. LLM routing arises because inference now happens over heterogeneous model pools rather than over one universal model. The field can be compared along axes of routing granularity, context representation, supervision, generalization, and overhead. The benchmark source papers show that routing claims are inseparable from the data regimes in which they are evaluated. Finally, broader related work reveals that the three focal papers are not isolated contributions, but part of a larger movement from cost-aware switching toward richer contextual and sequential routing policies.

Chapter 3

RouteLLM: Preference-Based Binary Routing

3.1 Motivation and Problem Setting

RouteLLM addresses the simplest practically important routing scenario: a strong model and a weak model are both available, and the system must decide which one should answer a given query. The strong model offers better expected quality but higher cost. The weak model offers lower cost but lower reliability. In many deployments this setting is realistic because a premium model serves as fallback while easy traffic is offloaded to a cheaper alternative. This makes RouteLLM especially relevant for stable strong-weak pairs and for environments in which routing overhead must remain low.

The paper’s contribution is to formulate this setting as preference-based binary routing. Rather than asking the router to generate answers, the method predicts whether the strong model is likely to outperform the weak model on the current query. This is attractive because it turns routing into a lightweight query-level prediction problem while still exposing an interpretable deployment policy.

3.2 Core Formulation

Let $D_{\text{pref}} = \{(q, l_{s,w})\}$ denote preference data, where $l_{s,w}$ records the observed outcome of comparing a strong and weak model on query q . RouteLLM learns a win-prediction model by maximizing

$$\max_{\theta} \sum_{(q, l_{s,w}) \in D_{\text{pref}}} \log P_{\theta}(l_{s,w} \mid q). \quad (3.1)$$

In other words, the router is trained to estimate whether the strong model should be expected to win for a given query. Once this probability model is learned, routing is

implemented through a threshold α :

$$R^\alpha(q) = \begin{cases} M_{\text{weak}}, & P_\theta(\text{win}_s \mid q) < \alpha, \\ M_{\text{strong}}, & P_\theta(\text{win}_s \mid q) \geq \alpha. \end{cases} \quad (3.2)$$

This formulation is powerful precisely because it is simple. The router only estimates whether escalation is worthwhile, and the system designer can move the operating point by adjusting α . Lower thresholds bias the policy toward stronger quality, while higher thresholds bias it toward lower cost. The method is therefore well suited to deployment settings where control simplicity and tunable cost-quality trade-offs matter more than rich sequential reasoning.

3.3 Supervision and Data Construction

The core training signal comes from Chatbot Arena [15], where users compare responses from anonymous models and select a winner. This data source is significant because it measures comparative response quality rather than only correctness against a gold label. In practice, many LLM tasks are open-ended, and preference comparisons are often more aligned with deployment goals than exact supervision.

The paper also acknowledges that raw preference data is sparse. Only a small fraction of possible model-pair comparisons are directly observed. To address this, models are grouped into capability tiers, and the strong and weak classes are constructed from these tiers. Beyond this, RouteLLM supplements the preference data with two augmentation schemes. First, it uses golden-labeled tasks such as MMLU [16] to derive comparison labels from correctness. Second, it uses an LLM judge on open-ended data, following the growing literature on LLM-as-a-judge evaluation such as MT-Bench [17].

This data pipeline is one of the paper’s most important practical contributions. The routing problem is only as good as its supervision, and raw preference data alone is not enough to cover the relevant query space. By combining human preferences with targeted augmentation, the paper turns sparse pairwise feedback into a more usable training signal for deployment-oriented routing.

3.4 Router Families

RouteLLM does not propose only one router architecture. Instead, it studies several possible families. These include a similarity-weighted ranking method inspired by Bradley–Terry comparisons, a matrix-factorization method that treats query–model quality interactions as a low-rank structure, a BERT-based classifier, and a causal-LLM classifier. This breadth matters because it shows that the paper’s main idea is not tied to one archi-

texture. The unifying principle is the use of preference supervision to estimate whether a premium model is worth calling.

The similarity-weighted ranking method is attractive when one wants minimal training overhead and strong locality-based intuition. The matrix-factorization approach is more structural: it assumes that a latent query–model score surface can be learned. The BERT and causal-LLM variants increase capacity and exploit richer textual representations, but they also rely more heavily on sufficient training data. The main trade-off across these router families lies in the balance between model capacity, supervision efficiency, and routing overhead.

Table 3.1: RouteLLM router families and their design trade-offs.

Router family	Main idea	Relative complexity	Main advantage / risk
Similarity-weighted ranking	Reuse nearby preference examples through embedding similarity	Low	Very lightweight; may be sensitive to dataset coverage
Matrix factorization	Learn latent query–model interaction scores	Medium	Captures structure beyond nearest-neighbor similarity; still relatively compact
BERT classifier	Predict strong-vs.-weak advantage from contextualized query representation	Medium to high	Strong textual encoding; depends more on sufficient supervision
Causal-LLM classifier	Use an instruction-tuned LLM as the router itself	High	High capacity and transfer potential; higher inference and training overhead

3.5 Architectural Instantiations in More Detail

The paper’s value lies partly in the fact that it does not reduce routing to a single implementation. The similarity-weighted ranking model is effectively a local estimator: it assumes that nearby preference examples contain enough information to predict whether the strong model will beat the weak model on a new query. The matrix-factorization model is more global. It assumes that query–model interactions have a latent low-rank structure, which makes it possible to interpolate beyond directly observed comparisons. The learned router is therefore not simply memorizing pairs, but inferring a smoother score surface over queries and model classes.

The classifier family provides a different perspective. In the BERT router, the query is encoded with a contextualized text encoder based on BERT [29] and the win probability is predicted by a lightweight classification head:

$$P_{\theta}(\text{win}_s \mid q) = \sigma(Wh_{\text{CLS}} + b), \quad (3.3)$$

where h_{CLS} is the pooled query representation. This formulation matters because it makes the router a standard supervised text classifier, which benefits from transfer from pre-trained encoders but also depends strongly on the training distribution. The causal-LLM router goes one step further: it treats routing itself as an instruction-following problem, allowing a higher-capacity model to map the query to a routing judgment. In deployment terms, RouteLLM therefore explores a continuum from extremely cheap local routing to more expressive but more expensive learned routing.

3.6 Benchmarks and Evaluation

For evaluation, RouteLLM uses several complementary benchmarks. MMLU measures broad multitask knowledge, GSM8K [18] measures mathematical reasoning, and MT-Bench measures conversational quality. These are deliberately different regimes. A router that works only on one style of task would be much less convincing than one that can preserve cost-quality trade-offs across broad knowledge, dialogue, and reasoning benchmarks.

The paper defines performance gap recovered as

$$\text{PGR}(M_{R^\alpha}) = \frac{r(M_{R^\alpha}) - r(M_w)}{r(M_s) - r(M_w)}, \quad (3.4)$$

where $r(M_{R^\alpha})$ is the routed system’s performance and $r(M_s)$ and $r(M_w)$ are the performances of the strong and weak models. APGR then aggregates the recovered gap across the strong-call curve, while CPT measures how many strong-model calls are needed to achieve a target PGR. These metrics are good examples of evaluation designed for deployment policy rather than for raw classification accuracy.

3.7 Experimental Findings Across Benchmarks

The paper’s experimental results are more informative when read benchmark by benchmark rather than only through one average number. On MT-Bench, routers trained only on Arena data already perform well because the training signal and evaluation style are both rooted in open-ended judged responses. The best Arena-only APGR comes from similarity-weighted ranking, while dataset augmentation with judge-generated com-

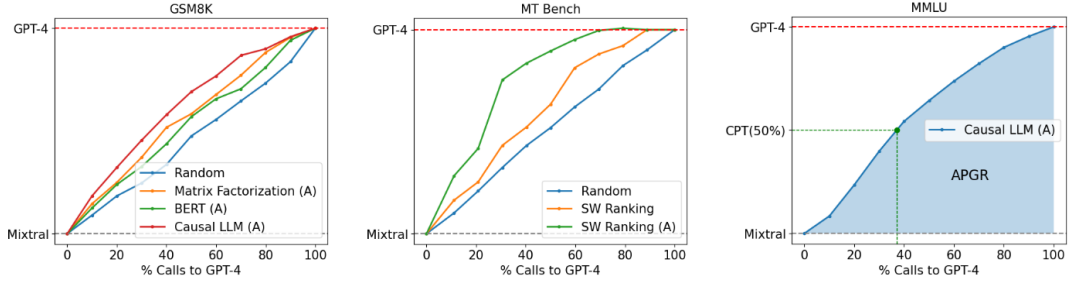


Figure 3.1: RouteLLM evaluates routers by the relationship between strong-model call rate and recovered performance. The figure is redrawn by the author based on RouteLLM [1].

parisons allows matrix factorization to perform substantially better. On MMLU and GSM8K, by contrast, Arena-only training is much less effective because these benchmarks are farther from the conversational distribution of the preference data.

The paper therefore makes an important empirical point: the effectiveness of a router depends not only on the router architecture, but also on how well the supervision resembles the evaluation distribution. On MMLU, even a relatively small amount of in-domain golden-labeled augmentation materially improves routing. On GSM8K, augmentation with judge-created labels substantially changes the picture and makes the stronger classifier family more competitive. In other words, RouteLLM is not a proof that one universal router can solve every routing setting. It is a proof that preference-based routing becomes much more effective when the supervision distribution is aligned with the target workload.

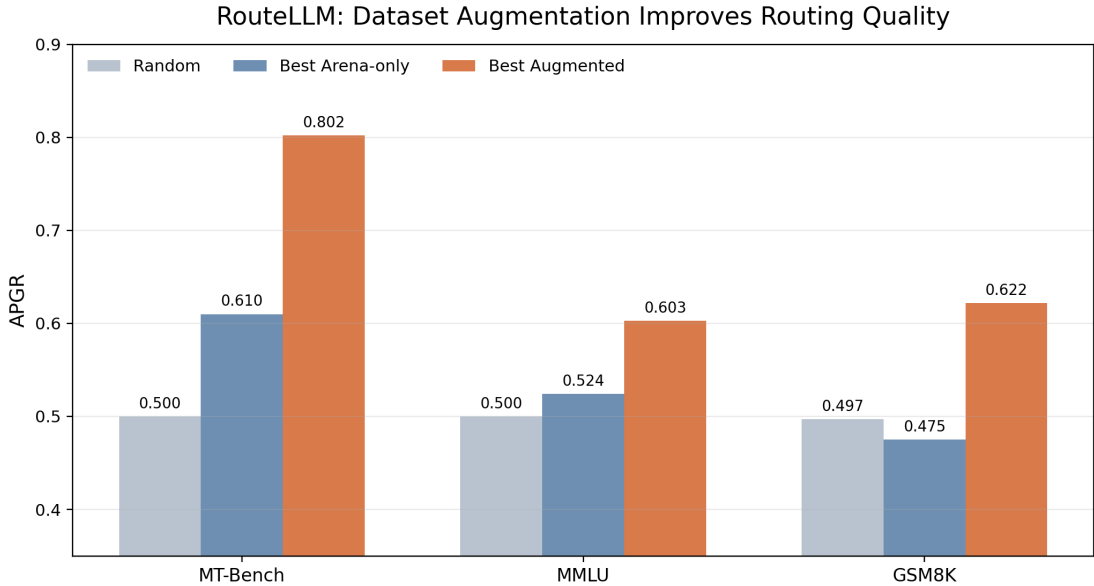


Figure 3.2: Selected APGR values from RouteLLM show that benchmark-aligned data augmentation substantially improves routing quality. The chart is redrawn by the author based on results reported in RouteLLM [1].

Table 3.2: Selected RouteLLM results reported across the three main benchmarks.

Benchmark	Random APGR	Best Arena-only result	Best augmented result	Main reading of the result
MT-Bench	0.500	SW Ranking: 0.610	Matrix Factorization on Arena+Judge: 0.802	Open-ended judged tasks are well matched to preference supervision and benefit further from label augmentation
MMLU	0.500	Matrix Factorization: 0.524	SW Ranking on Arena+Gold: 0.603	A small amount of in-domain gold supervision materially improves routing on academic knowledge tasks
GSM8K	0.497	SW Ranking: 0.475	Causal LLM on Arena+Judge: 0.622	Hard reasoning tasks require better-aligned supervision and often benefit from higher-capacity routers

3.8 Adaptation Across Model Pairs

One of the most practically interesting parts of RouteLLM is its cross-pair evaluation. After training routers on one strong-weak configuration, the authors test them on different model pairs without retraining, including Claude-family and Llama-family pairs. The reported results remain substantially better than random routing. This matters because deployment model pools change quickly. If a router only worked for the exact strong and weak models used during training, it would be much less useful in practice.

The paper interprets this finding as evidence that the routers are learning something about *query difficulty* rather than memorizing one pairwise comparison table. That interpretation is plausible, although it should be treated carefully. What transfers is not a universal notion of difficulty, but a notion of when a stronger model is likely to be worth its cost under a strong-weak distinction. This is still narrower than the generalization target pursued by GraphRouter and Router-R1, but it is already an important step beyond fixed heuristics.

3.9 Cost Savings, Overhead, and Deployment Interpretation

RouteLLM also studies two questions that are often neglected in purely algorithmic papers. The first is whether routing produces meaningful cost savings relative to always calling the strongest model. The second is whether the router itself adds enough overhead to undermine those savings. The reported cost-saving ratios show that the savings can be substantial, especially on MT-Bench-like workloads where open-ended preference signals align well with the task. At the same time, the measured routing overhead is small relative to premium-model generation cost, even for higher-capacity routers.

These observations matter because they sharpen the deployment argument. A router can be theoretically elegant but practically useless if it is too expensive or too slow to run. RouteLLM demonstrates the opposite: the routing layer is cheap enough that the dominant cost remains the premium model itself. This is one reason why the paper is an effective starting point in the routing literature. It pairs a clear learning formulation with an explicitly deployment-aware cost analysis.

3.10 Strengths

RouteLLM’s main strength is that it solves a real deployment problem with a minimal decision space and interpretable operating points. It is easy to understand, relatively easy to train, and directly exposes the quality-cost trade-off through the threshold parameter.

Table 3.3: RouteLLM deployment-oriented observations reported in the paper.

Aspect	Reported observation	Why it matters for deployment
Cost savings over always using GPT-4	Up to about $3.66\times$ on MT-Bench at the chosen operating point, with smaller but still meaningful gains on MMLU and GSM8K	Savings are largest when the router can reliably avoid premium-model calls on easy queries
Cross-model adaptation	Strong results remain when the routed pair is changed to Claude-family or Llama-family strong-weak pairs	The router captures transferable query-level escalation signals instead of only memorizing one fixed pair
Router overhead	The paper reports router overhead as small relative to premium-model generation cost, with the most expensive router still adding only a minor fraction of total cost	Routing is practically viable only if the policy layer is much cheaper than the premium model it helps avoid

For systems that primarily need premium-model escalation, this is a strong design. It is also comparatively light in runtime complexity: routing is a single pre-answer decision rather than a long control loop. In comparative terms, RouteLLM shows that even constrained routers can recover much of strong-model quality when the deployment scenario is well matched to the method.

3.11 Limitations

Its limitations, however, are structural. First, it is intrinsically binary and assumes that the routing problem can be reduced to a stable strong-weak distinction. Second, it makes a one-shot decision before observing any intermediate output, so it cannot revise its choice after reasoning begins. Third, it relies primarily on query-level signals and therefore cannot naturally exploit complementary strengths across several models. These limits are not unique to RouteLLM; they appear in different forms across many early routing systems such as FrugalGPT [4], HybridLLM [5], and meta-modeling approaches to cost-effective model choice [8]. As a result, the method is best viewed as intentionally narrow rather than generally complete.

3.12 Position in the Evolution of LLM Routing

Within the broader literature, RouteLLM represents a clean starting point for modern routing research. It demonstrates that routing can be learned rather than hand-designed, and that preference supervision provides a viable signal for deployment-oriented model selection. At the same time, it occupies the earliest and most constrained point in the design space examined in this report. The limitations of binary, query-only, one-shot routing motivate the shift toward richer contextual routers such as GraphRouter.

3.13 Summary

RouteLLM is best understood as a high-quality solution to a deliberately narrow routing problem. Its central insight is that preference data can support effective binary escalation policies, especially when benchmark-aligned augmentation is available. Its practical significance lies in the fact that it combines interpretable control, real cost savings, and modest routing overhead. Its limitations are equally instructive: once the routing problem is no longer binary, once richer task context matters, or once routing must unfold over several rounds, the RouteLLM formulation becomes too restrictive. These are precisely the pressures that motivate GraphRouter and Router-R1.

Chapter 4

GraphRouter: Graph-Based Multi-Model Selection

4.1 Problem Setting and Motivation

GraphRouter begins from a more realistic deployment setting than RouteLLM. Instead of choosing between one strong model and one weak model, a system may need to choose among many models with different sizes, families, costs, and specializations. Furthermore, the same model may behave differently across tasks, and some smaller models may outperform larger ones on specific queries. These observations suggest that LLM routing should be treated as a contextual recommendation problem rather than a plain query-classification problem.

The paper’s core claim is that routing quality depends on modeling relationships among tasks, queries, and models. If model identity is represented only as an index, the router cannot easily generalize to newly introduced models, nor can it exploit richer contextual structure. In this sense, GraphRouter does not merely change the decision rule used by earlier routers. It changes the representation space of routing itself by treating interaction records as structured relational evidence.

4.2 Graph Construction

GraphRouter constructs three kinds of nodes: task nodes, query nodes, and LLM nodes. It then connects them using typed edges. Task–query edges represent task membership. Query–LLM edges encode observed interaction attributes such as effect and cost. The resulting graph is a structured record of prior routing experience.

This representation is significant because it transforms routing into a relational prediction problem. The router is no longer simply learning from queries in isolation. Instead, it learns how tasks, queries, and LLMs co-vary. In practical terms, this means the system

can infer that certain models are broadly strong on mathematical tasks, others are efficient on summarization, and still others should only be selected when the performance target is especially high. The graph is therefore not only a modeling device but also a commitment to a richer representation of routing experience.

4.3 Inductive Initialization and Heterogeneous GNN

The model’s inductive design is one of its most important features. Query nodes are initialized from textual encoders. Task nodes and LLM nodes are initialized from generated descriptions rather than only from one-hot identities. For LLM nodes, the description includes strengths, cost, and context information. This allows newly added models to receive meaningful initial embeddings without requiring the router to treat them as entirely unknown IDs.

After initialization, the model performs type-aware aggregation using a heterogeneous graph neural network. A simplified task-node update can be written as

$$h_t^{(l)} = U_t^{(l)} \text{CONCAT}(\text{MEAN}\{\text{ReLU}(W_t^{(l)} h_q^{(l-1)}), \\ q \in N(t)\}, h_t^{(l-1)}). \quad (4.1)$$

After several layers, the task and query states are combined:

$$h_{qt}^{(L)} = \text{MLP}\left(\text{CONCAT}(h_t^{(L)}, h_q^{(L)})\right), \quad (4.2)$$

and routing is implemented as an edge-prediction problem:

$$\hat{y} = \arg \max_m \text{EdgePred}(h_{qt}^{(L)}, h_m^{(L)}). \quad (4.3)$$

These equations make clear that GraphRouter is learning a compatibility function over structured state rather than a flat classifier over query text. The heterogeneous message-passing design is closest in spirit to graph neural network families such as GCN [30], GraphSAGE [31], and GAT [32], but adapted to typed task–query–LLM relations. The inductive initialization strategy is especially important because it gives newly introduced models a meaningful starting representation from textual descriptions, which matters in an ecosystem where model pools change rapidly.

4.4 Datasets and Experimental Setting

GraphRouter is evaluated on a multi-task dataset built from several benchmarks. Alpaca [19], itself derived from self-instruct style data generation [33], provides instruction-following hybrid QA. GSM8K provides mathematical reasoning. SQuAD [20] provides

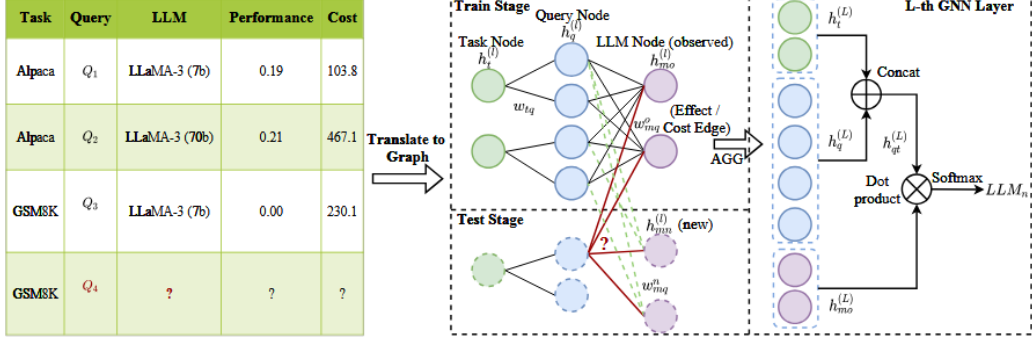


Figure 4.1: GraphRouter transforms interaction records into a heterogeneous graph and performs typed aggregation before edge prediction. The figure is redrawn by the author based on GraphRouter [2].

reading comprehension. Multi-News [21] provides summarization. The paper then collects responses from ten LLMs of different sizes and prices and computes effect and cost for each interaction. This yields a richer setting than binary routing because the router must choose among many models and many tasks simultaneously.

To compare methods under different deployment priorities, the paper uses a weighted reward:

$$\text{Reward} = \alpha \cdot \text{Performance} - \beta \cdot \text{Cost}, \quad (4.4)$$

with different (α, β) settings corresponding to performance-first, balanced, and cost-first scenarios. This framework is valuable because it mirrors the fact that real systems may optimize different objectives depending on workload conditions and service-level goals.

4.5 Baselines and Evaluation Scenarios

GraphRouter’s baselines deserve attention because they show what the paper considers a fair comparison. The simplest baselines always choose the largest or smallest model. A prompt-based LLM baseline chooses a model by reasoning over the query and candidate descriptions directly in prompting. The learned baselines include HybridLLM, Frugal-GPT, and C2MAB-V, which respectively represent binary routing, score-prediction routing, and bandit-style online selection. Finally, the paper defines an oracle upper bound in which each query is assigned to the truly best LLM. This oracle is important because it prevents GraphRouter from being judged only against weak baselines. It shows how much headroom remains even after contextual graph modeling.

The three evaluation scenarios are also conceptually useful. In the performance-first setting, quality dominates and cost matters little. In the balanced setting, effect and price are weighted more symmetrically. In the cost-first setting, the deployment policy is encouraged to sacrifice some quality to reduce spending. These scenarios are not only presentation choices. They make explicit that routing is not optimizing one timeless

Table 4.1: Compact view of GraphRouter’s experimental setting.

Component	Value reported in the paper	Comparative significance
Task families	Alpaca, GSM8K, SQuAD, Multi-News	The benchmark mix covers instruction following, reasoning, reading comprehension, and summarization rather than only one QA style
Interaction dataset size	600 cases per task in the standard setup	GraphRouter studies moderate multi-task interaction logs rather than the huge preference corpora used by RouteLLM
Candidate models	10 LLMs with different sizes and token costs	The routing problem is genuinely multi-model rather than binary
Optimization target	Weighted reward over performance and cost	Deployment priorities are encoded explicitly through scenario-dependent coefficients
Generalization setting	New-LLM setting with auxiliary few-shot interaction data	The paper treats unseen model incorporation as a first-class evaluation target

objective, but a tunable deployment preference.

4.6 Results and Interpretation

GraphRouter consistently outperforms prior baselines such as HybridLLM, FrugalGPT, and C2MAB-V across the studied settings. The paper reports a minimum performance improvement of 12.3% in the considered effect-cost scenarios. It also shows stronger generalization when previously unseen LLMs are introduced at test time.

These results are meaningful for two reasons. First, they suggest that explicit relational structure improves routing quality beyond what one can obtain from query-only classifiers. Second, they show that inductive model representations are useful in a rapidly changing ecosystem where new models arrive frequently. A router that requires expensive retraining every time a new model appears is much less practical than one that can form a useful prior from a description and a few examples.

4.7 Generalization to Newly Introduced LLMs

The paper’s most distinctive experimental claim is that GraphRouter handles new LLMs more effectively than earlier routers. This is where the inductive design becomes decisive.

Table 4.2: Selected GraphRouter results under different deployment scenarios.

Scenario	Strongest non-Graph baseline	GraphRouter reward	Oracle reward	Reading of the result
Performance first	FrugalGPT: 0.517	0.539	0.588	GraphRouter improves pure-quality routing by exploiting contextual structure rather than only cost heuristics
Balanced	C2MAB-V: 0.196	0.209	0.231	Gains remain when quality and cost are jointly weighted, which is the most realistic deployment regime in the paper
Cost first	FrugalGPT: 0.057	0.064	0.072	Even in strongly cost-sensitive settings, contextual relational modeling remains beneficial

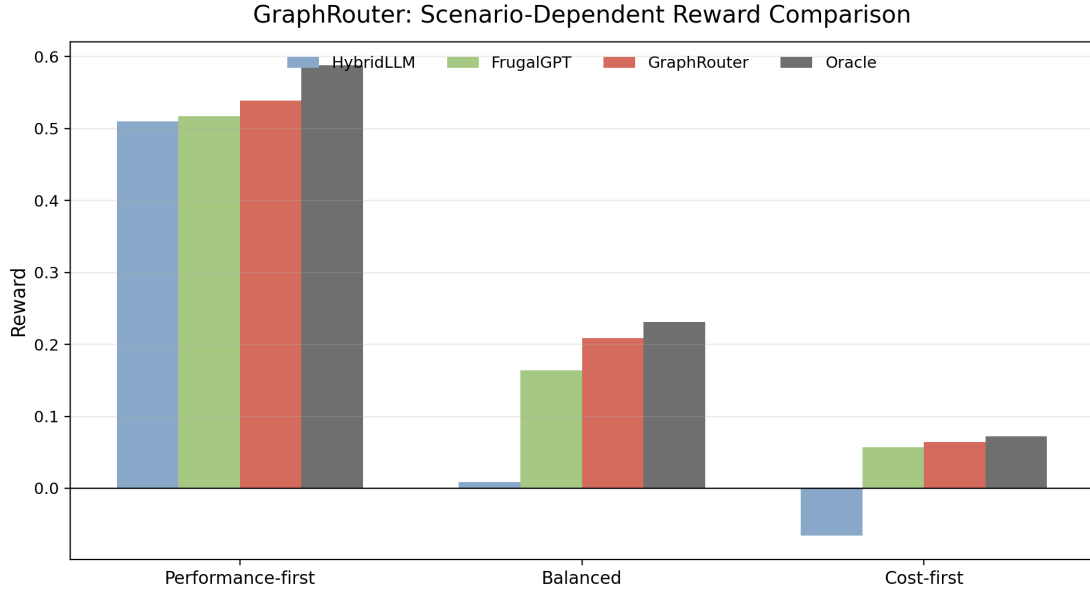


Figure 4.2: GraphRouter’s advantage persists across performance-first, balanced, and cost-first scenarios. The chart is redrawn by the author based on results reported in GraphRouter [2].

Rather than learning only a fixed mapping from queries to known model identifiers, GraphRouter can initialize new LLM nodes from textual descriptions and a small amount of auxiliary interaction data. In the few-shot new-LLM setting reported by the paper, GraphRouter retains strong reward while dramatically reducing the time cost relative to baselines that must retrain more heavily.

This finding should be interpreted carefully. GraphRouter does not solve new-model generalization for free. It still needs descriptions, auxiliary examples, and a graph structure in which the new model can be embedded. But compared with static-ID routing, it is a qualitatively better answer to the realistic problem that candidate model pools change over time.

4.8 Ablations and Extended Experiments

GraphRouter also reports useful ablations that are easy to overlook. The reward is highest when the hidden size is moderate rather than maximal, and when the heterogeneous GNN uses two layers rather than a much deeper stack. This matters because it suggests that relational routing benefits from enough graph capacity to model context, but not from unbounded graph depth. Over-smoothing and unnecessary complexity are real risks. The paper further extends the setup to larger datasets and larger model pools, including additional tasks and LLMs, and still reports strong gains over the baselines.

These additional experiments strengthen the paper in two ways. First, they show that the graph formulation is not an artifact of one carefully chosen benchmark mix. Second,

Table 4.3: GraphRouter in the few-shot new-LLM setting.

Method	Reward	Time cost	Interpretation
HybridLLM	0.010	273.45	Weak transfer and high cost when new models must be integrated
FrugalGPT	0.171	63.15	Better than HybridLLM, but still expensive relative to graph induction
C2MAB-V	0.189	542.25	Stronger reward than simple baselines, but extremely costly in this setting
GraphRouter (few-shots)	0.207	3.00	Best efficiency among reported methods while also improving reward
GraphRouter (trained on full pool)	0.219	30.00	Slightly stronger reward, but much more expensive than the few-shot inductive variant

they suggest that GraphRouter occupies a middle ground between lightweight heuristics and very heavy sequential policies. It is more demanding than RouteLLM, but still much easier to deploy than a multi-round reinforcement-learned router.

4.9 Strengths

GraphRouter is a clear advance over binary routing methods. It expands the action space to many models, enriches the represented state, and handles new models more gracefully. Its main strengths are the ability to leverage relations across tasks, queries, and models, the use of inductive descriptions for unseen models, and the move beyond query-only signals. Relative to RouteLLM, it shows that better routing may come not from more aggressive control, but from better representation of the routing context.

4.10 Limitations

GraphRouter’s gains come with meaningful costs and assumptions. It depends on sufficiently rich interaction logs, and the graph itself must be constructed, updated, and maintained. Sparsity and cold-start issues may remain when tasks or models have limited history. Generalization to unseen models also depends on the quality of the textual descriptions used for inductive initialization, which introduces an additional source of variability. Most importantly, GraphRouter still performs one-shot selection. It chooses

one LLM for one query and stops. If a query would benefit from decomposition or staged evidence gathering, GraphRouter cannot revise its decision during execution.

4.11 Position in the Evolution of LLM Routing

GraphRouter marks a major expansion of the routing design space. It moves routing beyond binary choice by enriching context representation and by treating model selection as relational inference across a heterogeneous ecosystem. At the same time, it still treats routing as a one-time placement decision. This makes it a strong intermediate point in the literature: more expressive than RouteLLM, but less adaptive than the sequential policies represented by Router-R1.

4.12 Summary

GraphRouter’s core contribution is representational rather than merely algorithmic. It argues that routing quality improves when tasks, queries, and models are modeled jointly rather than in isolation, and it backs that claim with scenario-based evaluation, few-shot new-model transfer, and informative ablations. Its main limitation is that all of this richer context still culminates in a one-shot decision. For workloads that truly require staged evidence gathering or repeated routing, the design remains too static. That limitation opens the door to Router-R1.

Chapter 5

Router-R1: Agentic Multi-Round Routing

5.1 Why One-Shot Routing Is Sometimes Not Enough

Router-R1 starts from a stronger claim than the previous two papers: one-shot routing is often insufficient for difficult reasoning tasks, especially multi-hop question answering, compositional reasoning, and staged evidence collection. In these settings, the correct action is not simply to pick the globally strongest model or the single best model for the raw query. Instead, the system may need to decide what information is missing, which model is suitable for obtaining it, and whether further model calls are needed after new information is received.

This reframes routing as a sequential control problem. The router is no longer just a classifier external to the answering process. It becomes an active policy that can plan, act, observe, and act again. Importantly, Router-R1 does not claim that all routing should be multi-round. Rather, it argues that richer control is justified in workloads where decomposition and revision materially affect answer quality.

5.2 Sequential RL Formulation

Router-R1 instantiates the router itself as a capable LLM and trains it with reinforcement learning. The paper writes the optimization objective as

$$\max_{\pi} \mathbb{E}_{x \sim D, y \sim \pi(\cdot | x; P)} \left[r_{\phi}(x, y) - \beta \log \frac{\pi(y | x; P)}{\pi_{\text{ref}}(y | x; P)} \right], \quad (5.1)$$

where π is the policy LLM, π_{ref} is a reference policy, and P is the routing pool. The important point is that the generated output y is not just a plain answer. It may include internal reasoning and several external routing actions. Thus the learned policy

coordinates a structured trajectory rather than a single label.

The reward function is defined as

$$r_\phi(x, y) = R_{\text{format}} + (1 - \alpha)R_{\text{outcome}} + \alpha R_{\text{cost}}, \quad (5.2)$$

where the cost reward is inversely related to the expense of model usage:

$$R_{\text{cost}} \propto -m(P_{\text{LLM}}) \cdot T_{\text{out}}. \quad (5.3)$$

This design explicitly embeds resource cost into the policy objective. The policy is not trained only to generate good answers; it is trained to coordinate external model usage under a cost-quality objective. This distinction is what separates Router-R1 from ordinary answer-generation fine-tuning.

5.3 Reward Curation

The reward design is one of the paper’s most practically important choices. Router-R1 does not rely on a learned reward model. Instead, it uses a lightweight rule-based reward composed of format, outcome, and cost terms. The format term enforces that the model follows the structured routing protocol. In simplified form, the paper uses a piecewise reward

$$R_{\text{format}} = \begin{cases} -1, & \text{if the rollout violates the required interaction format,} \\ 0, & \text{otherwise.} \end{cases} \quad (5.4)$$

The outcome term is tied to final answer correctness, typically using exact match against the gold answer. The cost term penalizes expensive routed-model usage, making it possible to tune the behavior of the policy by changing the coefficient α in Equation 5.2.

This reward structure matters for two reasons. First, it avoids the engineering complexity of learned reward-model pipelines. Second, it makes the training objective legible enough to analyze. The router is explicitly being optimized to produce well-formed action traces, correct answers, and acceptable cost behavior. This clarity is valuable because one of the main risks in multi-round routing is that control complexity can make optimization and debugging opaque.

5.4 Action Protocol and Prompt Structure

The method uses a structured action protocol based on tags such as `<think>`, `<search>`, `<info>`, and `<answer>`. The policy first reasons internally inside a `<think>` block. If more information is needed, it emits a `<search>` action that specifies which candidate

model to call and what sub-query to ask. The returned result is inserted as `<info>`, after which the policy can reason again, call another model, or terminate with `<answer>`.

This protocol matters for three reasons. First, it makes the routing trajectory interpretable. Second, it tightly couples reasoning and resource allocation. Third, it allows the policy to generalize to new models by conditioning on textual model descriptors rather than memorizing a fixed routing pool. Compared with earlier routers, Router-R1 expands the router’s action space into planning, external querying, and revision.

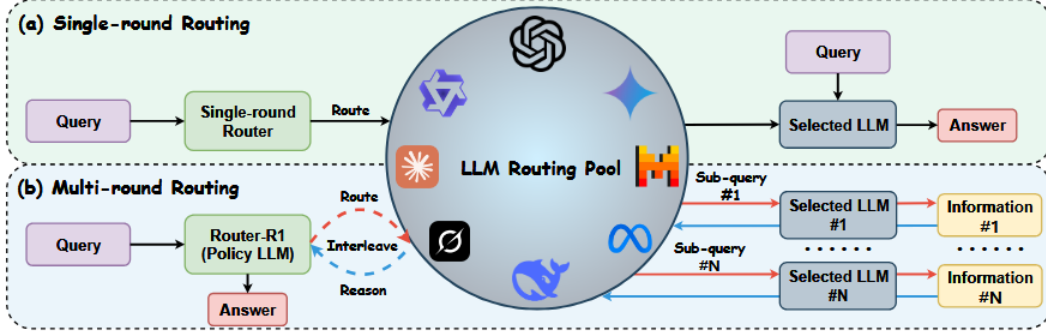


Figure 5.1: Router-R1 turns routing into a multi-round procedure in which the policy alternates between reasoning and external model invocation. The figure is redrawn by the author based on Router-R1 [3].

5.5 Training Prompt and Interaction Loop

The training prompt used by Router-R1 is itself a conceptual contribution. The router is instructed to reason inside `<think>` tags before every external call, to explain why additional information is needed, and to choose a candidate LLM based on its brief descriptor. If external information is needed, the router must issue a structured `<search>` command specifying both the chosen model and the sub-query. The returned response is wrapped in `<info>` and appended to the evolving context, after which the policy can continue reasoning.

This interaction loop effectively turns routing into a tool-using reasoning process. The router is not only selecting a model; it is deciding what information is missing, which model is best suited to provide that information, and whether the current state is sufficient to answer. The policy therefore integrates decomposition, routing, and aggregation into one learned control loop. That is a substantial conceptual shift from both RouteLLM and GraphRouter.

5.6 Benchmarks and Results

Router-R1 is evaluated on a broad QA suite. The general-QA benchmarks include Natural Questions [22], TriviaQA [23], and PopQA [24]. The multi-hop benchmarks include HotpotQA [25], 2WikiMultiHopQA [26], MuSiQue [27], and Bamboogle [28]. This suite is well matched to the paper’s thesis: if multi-round routing is genuinely useful, it should help more on multi-hop and compositional tasks than on direct factoid QA.

The paper reports that Router-R1 achieves the best average exact-match performance among the compared systems on both Qwen- and LLaMA-based backbones. It also shows strong out-of-domain generalization despite training on a relatively modest joint dataset. The qualitative explanation offered by the paper is persuasive: Router-R1 can decompose complex tasks into several model interactions, exploiting complementary strengths across the routing pool instead of relying on one-shot selection.

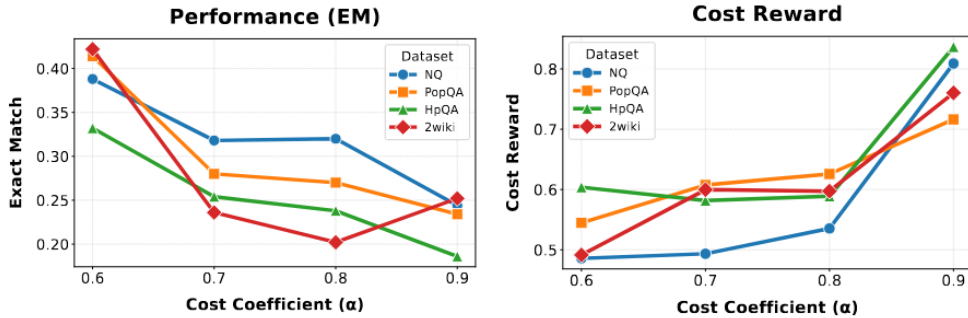


Figure 5.2: Router-R1 explicitly studies the impact of the cost coefficient α on answer quality and cost reward. The figure is redrawn by the author based on Router-R1 [3].

The cost-coefficient experiments are especially relevant because they show that the policy learns a tunable efficiency-quality frontier rather than merely maximizing correctness. As α increases, the policy becomes more cost-sensitive and tends to favor cheaper actions and smaller models. This improves the cost reward but usually decreases answer accuracy. The result clarifies both the promise and the practical burden of multi-round routing: richer control can be learned, but it remains expensive enough to require explicit tuning.

5.7 Baselines and Experimental Protocol

Router-R1 is evaluated against an unusually broad baseline suite. The paper includes direct inference, chain-of-thought prompting [11], supervised fine-tuning, retrieval-augmented generation, Search-R1 [13], and a large family of query-based LLM routers including prompt-based selection, KNN and MLP routers, BERT Router, RouterDC, and GraphRouter. This is important because Router-R1 is not being compared only to weak baselines or

only to other sequential systems. It is being positioned as a general answer to a broad set of routing and reasoning baselines.

The implementation details also deserve attention. The paper trains Router-R1 on a joint 14K-sample dataset constructed from Natural Questions and HotpotQA, uses a maximum of four routing steps, and evaluates on seven QA datasets with consistent sampling. It employs relatively small base models, namely Qwen2.5-3B-Instruct and Llama-3.2-3B-Instruct, while drawing on a routing pool of stronger external models. This setup makes the paper’s claim more compelling: the improvement is not merely due to scaling the base model, but to better coordination across models.

Table 5.1: Compact view of Router-R1’s training and evaluation protocol.

Component	Setting reported in the paper	Why it matters
Base policy models	Qwen2.5-3B-Instruct and Llama-3.2-3B-Instruct	Improvements are attributed to routing policy learning rather than only to a very large base model
Training data	Joint 14K-sample set from Natural Questions and HotpotQA	The paper claims strong transfer despite training on a modest in-domain set
Maximum routing depth	Four routing steps per query	Multi-round control is bounded, which keeps the action space manageable
Routing pool	Six candidate LLMs from different families and sizes	The policy must reason over heterogeneous external capabilities rather than over one homogeneous backend
Evaluation suite	Seven QA datasets spanning general and multi-hop QA	The benefits of sequential routing can be tested on both simple and compositional reasoning workloads

5.8 Main Results in Comparative Perspective

The main results show that Router-R1 consistently outperforms both basic baselines and query-based routers across seven QA datasets. This is most convincing when one compares the *average* exact-match results across backbone models. Direct inference and standard prompting are much worse. Search-R1 is a stronger comparator because it also treats inference as a multi-step process, yet Router-R1 still performs better on average. Compared with GraphRouter and RouterDC, the gains suggest that repeated reasoning and repeated routing are beneficial when the task itself has multi-step structure.

Table 5.2: Selected average exact-match results reported for Router-R1.

Base model	Direct	Search-R1	GraphRouter	Router-R1
Qwen2.5-3B-Instruct	0.135	0.291	0.297	0.416
Llama-3.2-3B-Instruct	0.150	0.311	0.300	0.409

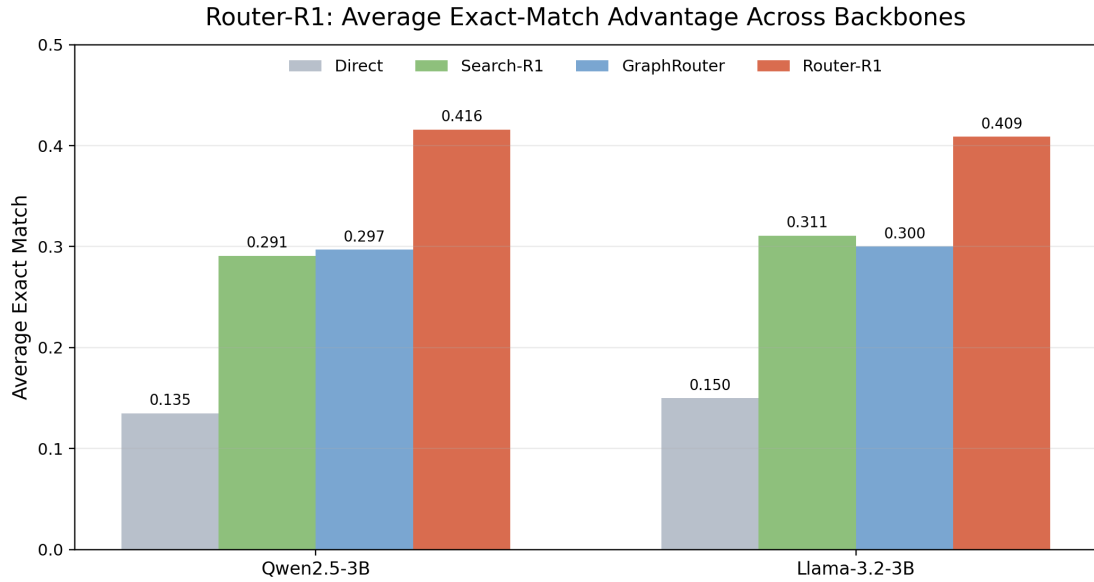


Figure 5.3: Router-R1 improves average exact-match over direct inference, Search-R1, and GraphRouter on both evaluated backbones. The chart is redrawn by the author based on results reported in Router-R1 [3].

The magnitude of these gains should not be overinterpreted as a universal guarantee that multi-round routing is always superior. The paper’s datasets are chosen precisely because they are favorable to staged reasoning, and this is one reason the results are meaningful. They show that when the task structure contains missing-evidence problems or compositional dependencies, one-shot routing may be too limited. They do not show that multi-round routing should replace simpler routers in every setting.

5.9 Cost-Aware Routing and Pareto Trade-offs

The paper’s cost analysis is more detailed than a simple claim that “cost matters.” By changing the coefficient α , the authors trace a family of policies along a performance–cost frontier. When $\alpha = 0.0$, the policy is performance-oriented and uses more expensive routing behavior. As α increases, the policy moves toward cheaper decisions. The paper reports that $\alpha = 0.6$ often gives a favorable balance, while much larger values produce sharper accuracy drops in exchange for large cost reductions.

This is conceptually important because it shows that Router-R1 is not only a reasoning system, but also a controllable deployment policy. It can be tuned toward accuracy or toward efficiency without changing the core architecture. That makes it closer to a family of policies than to one fixed router.

Table 5.3: Illustrative cost–accuracy trade-off for Router-R1 on Natural Questions.

α	Exact Match	Raw cost reward	Interpretation
0.0	0.388	150.6	Performance-oriented routing; highest accuracy among the reported settings, but also the highest cost
0.6	0.388	150.0	Similar accuracy with a slightly more disciplined cost behavior; presented by the paper as a strong balance point
0.7	0.318	32.3	Clear shift toward cheaper actions with visible accuracy loss
0.8	0.320	28.9	Very cost-conscious behavior with further performance sacrifice
0.9	0.244	5.5	Extremely aggressive cost minimization; the policy becomes much cheaper but substantially less accurate

5.10 Generalization to Unseen Candidate Models

Router-R1 also evaluates whether the learned policy can handle new candidate models introduced after training. The paper extends the routing pool with previously unseen models and supplies their descriptors at inference time without retraining the Router-R1 checkpoint. The resulting performance remains stable and in some cases slightly improves, suggesting that descriptor-conditioned routing is doing meaningful work rather than serving as a superficial prompt detail.

This experiment is especially important in comparison with earlier routers. RouteLLM generalizes across strong-weak pairs, and GraphRouter generalizes to new LLM nodes with inductive initialization, but Router-R1 adds sequential control on top of descriptor-based generalization. The result is not only that the router can score a new candidate. It can actively decide when and how to use that candidate during a reasoning trajectory.

Table 5.4: Average exact-match before and after adding unseen candidate LLMs to the routing pool.

Method (Qwen backbone)	Original pool	Extended pool	Interpretation
Prompt LLM	0.372	0.372	Almost no benefit from the added models
BERT Router	0.289	0.306	Some benefit, but still limited compared with stronger methods
GraphRouter	0.344	0.345	Stable transfer, but only modest improvement
Prompt LLM*	0.305	0.254	Decomposition alone does not guarantee robust unseen-model transfer
Router-R1	0.458	0.463	Sequential, descriptor-based routing benefits from added candidates without retraining

5.11 Behavioral Analysis and Descriptor Sensitivity

Beyond end-to-end accuracy, the paper also studies Router-R1’s behavior. On multi-hop datasets, the policy makes noticeably more external model calls than on general QA benchmarks. This is an encouraging sign because it suggests that the model is using external routing selectively rather than uniformly. The paper also reports rapid training convergence and shows that removing or weakening format rewards makes optimization less stable.

The descriptor-sensitivity analysis is also informative. When the model descriptors are compressed to shorter versions that preserve only key attributes, the performance changes only slightly. This implies that the router is not depending on verbose natural-language descriptions in a fragile way. It mainly needs a compact account of model capability and cost. For deployment, that is a practical advantage because model metadata can be simplified without destroying routing quality.

5.12 Strengths

Router-R1’s main strength is adaptive control during execution. It can revise decisions after observing intermediate information, decompose hard queries into sub-questions, and target different models for different stages of the same problem. This makes it especially well aligned with multi-hop and compositional tasks, where one-shot placement may be too coarse. Compared with RouteLLM and GraphRouter, Router-R1 is the method in this report that most directly turns routing into an explicit reasoning policy.

5.13 Limitations and Costs

The same flexibility introduces substantial costs. Multi-round trajectories lengthen the inference path and increase latency uncertainty. Errors can accumulate across rounds, especially if early reasoning or action generation is unstable. The method also depends on the reliability of a structured action protocol, which can make debugging and reproducibility more difficult than in simpler routers. Finally, the evaluation burden becomes heavier because performance now depends on both answer quality and the behavior of the intermediate control sequence. For these reasons, Router-R1 should be understood as the most adaptive design in this report, but also the one with the highest runtime and implementation burden.

5.14 Relation to the Broader RL and Reasoning Literature

Router-R1 belongs to a broader shift in LLM training. Earlier alignment methods such as RLHF [34] and DPO [35] focus mainly on improving the outputs of one model. Router-R1 instead optimizes a coordination policy over several models. Its formulation is closer to PPO-style policy optimization [36] and to recent reasoning-oriented RL systems such as DeepSeek-R1 [14] and Search-R1 [13]. What distinguishes it is that the “environment” includes a routing pool of external LLMs rather than only a search engine or internal reasoning traces.

5.15 Position in the Evolution of LLM Routing

Within the trajectory studied in this report, Router-R1 represents the most adaptive endpoint so far. It expands routing from one-shot selection to sequential policy control and thereby broadens both the capabilities and the costs of the router. Its importance lies not merely in stronger reasoning, but in redefining what routing is allowed to do.

5.16 Cross-Paper Comparison

At this point, the progression across the three papers can be stated clearly. RouteLLM is a binary escalation policy. GraphRouter is a relational multi-model recommender. Router-R1 is an agentic multi-round routing policy. Table 5.5 summarizes the main differences under the comparative framework established in Chapter 2.

Table 5.5: Comparison of the three focal papers.

Dimension	RouteLLM	GraphRouter	Router-R1
Routing granularity	Binary strong-vs-weak routing	Multi-model one-shot selection	Multi-round sequential coordination
Main supervision	Preference labels and augmentation	Interaction records with effect and cost	Reinforcement learning with outcome and cost rewards
Context representation	Mostly query-centric	Task-query-model graph	Query, model descriptors, and evolving interaction history
Generalization target	New strong-weak model pairs	New models and tasks	New candidate models via descriptors
Main decision mode	Thresholded escalation	Relational compatibility prediction	Sequential planning and action selection
Main benefit	Very low overhead with interpretable control	Richer context and better multi-model generalization	Highest adaptivity for compositional tasks
Main limitation	Binary and pre-answer only	Still one-shot after placement	Higher training and inference complexity

This table shows that the progression is not merely an increase in model complexity. Instead, it is an expansion in what the routing policy is allowed to observe and decide. RouteLLM sees the query and predicts whether escalation is needed. GraphRouter sees structured relations among tasks, queries, and models. Router-R1 additionally sees the evolving trajectory of intermediate evidence and can revise its decisions accordingly.

5.17 Practical Design Trade-offs

The practical implication of this progression is that different routing paradigms are appropriate for different deployment regimes. RouteLLM is a good fit when overhead must be minimal and the system mainly needs premium-model escalation. GraphRouter is more appropriate when there are many models, many tasks, and frequent model turnover, so richer context representation becomes worth the added complexity. Router-R1 is most promising when the workload contains difficult, multi-step tasks for which several targeted model calls can significantly improve final quality.

At the same time, more adaptivity is not always better. Richer routers demand more data, more engineering effort, and more careful evaluation. The comparison therefore suggests a conditional conclusion rather than a universal ranking. Binary routing is sufficient when the deployment structure is stable and the decision space is simple. Contextual multi-model routing is justified when relational information materially improves generalization. Multi-round routing is justified when one-shot placement is too coarse for the task structure. The central trade-off is not only capability versus cost, but also representational richness versus control overhead.

5.18 Summary

Router-R1 is the most ambitious of the three focal papers. Its contribution is not only better answer quality on selected benchmarks, but a new definition of what routing can mean: reasoning, external model invocation, and aggregation inside one learned policy loop. The same expansion creates new burdens in reward design, action stability, latency, and reproducibility. This dual character is why Router-R1 is both the most forward-looking and the most demanding paper in this report.

Chapter 6

Cross-Paper Synthesis and Open Challenges

6.1 Revisiting the Three Evolutionary Axes

The three focal papers can now be compared more systematically. Along the axis of routing granularity, the progression is clear: RouteLLM solves binary escalation, GraphRouter solves multi-model one-shot placement, and Router-R1 solves multi-round sequential coordination. Along the axis of context representation, the evolution is from query-only signals, to relational task–query–model structure, to trajectory-aware state enriched with intermediate observations. Along the axis of decision adaptivity, the move is from thresholded one-shot routing, to context-informed one-shot routing, to a policy that can revise its own routing trajectory over time.

These three axes are useful because they explain both the gains and the costs of the literature’s evolution. Higher granularity expands the action space. Richer context expands the state space. Greater adaptivity expands the optimization problem. Each transition offers more expressive control, but also demands more data, more training machinery, and more evaluation discipline. This is the central comparative lesson of the report.

6.2 Supervision, Data Construction, and Labeling Burden

Another way to read the evolution is through the supervision problem. RouteLLM depends on preference comparisons, augmented by in-domain gold labels or LLM-judge labels. GraphRouter depends on interaction logs that tie tasks, queries, models, and costs together. Router-R1 depends on a reinforcement-learning setup with structured rollouts and a carefully curated rule-based reward. These are not merely different train-

ing choices. They define different assumptions about what data a deployment or research group can realistically collect.

This perspective helps explain why later methods are not automatically dominant. Preference data is easier to justify in many chatbot settings than graph-structured interaction logs. Graph-structured logs may be easier to obtain than stable RL training with structured action protocols. Therefore, the literature’s movement toward richer routing policies should not be mistaken for a simple linear improvement curve. It is better understood as a movement into more demanding supervision regimes.

6.3 Evaluation Regimes and Their Hidden Assumptions

The evaluation setups of the three papers are equally revealing. RouteLLM assumes that routing quality should be measured by how much strong-model performance can be recovered with fewer premium calls. GraphRouter assumes that routing should be judged by a scenario-dependent reward balancing performance and cost. Router-R1 assumes that exact-match or F1 performance on difficult QA tasks should be paired with cost-aware reward analysis and behavioral diagnostics.

These choices are reasonable within each paper, but they are not interchangeable. A router optimized for MT-Bench-style judged dialogue is not being asked the same question as a router optimized for HotpotQA-style multi-hop reasoning. As a result, a mature routing literature will likely need more standardized cross-setting benchmarks, or at least better mapping between evaluation regimes. RouterBench [10] is one step in this direction, but the problem is far from solved.

6.4 Deployment Suitability and Workload Matching

The three papers also suggest a practical decision framework for deployment. If the service has a stable premium fallback model, if the goal is mainly cost reduction on easy queries, and if the routing budget must stay very small, a RouteLLM-style binary router is often sufficient. If the service exposes a larger and more dynamic model pool, and if historical interaction data is available, GraphRouter-style contextual routing becomes attractive. If the workload is dominated by compositional tasks, multi-hop evidence needs, or staged problem decomposition, Router-R1-style sequential routing becomes plausible despite its overhead.

This workload-matching view is important because it prevents the literature from being read as a race toward maximum sophistication. The right router depends on the structure of the queries, the stability of the model pool, the available supervision, and

the latency budget. In practice, different workloads may justify different routers within the same service stack.

6.5 Open Problems

Several open problems remain. The first is latency-aware routing under concurrency. All three papers focus more directly on quality and monetary cost than on tail latency under realistic service pressure. The second is fairness and multi-tenant budgeting, since most experiments optimize per-query outcomes in isolation. The third is online adaptation: model prices, traffic distributions, and service conditions change over time, while current evaluations remain mostly offline. The fourth is cross-layer integration, because routing decisions in production may need to interact with queueing, batching, hardware availability, and caching rather than only with model descriptors and benchmark accuracy.

Table 6.1: Open problems suggested by the three focal papers.

Open problem	Why current papers do not fully solve it	Deployment implication
Latency-aware routing	Existing evaluations emphasize quality and cost more than tail latency under concurrent load	Routers may choose high-quality policies that are impractical under strict service-level objectives
Multi-tenant fairness	Most experiments optimize per-query outcomes in isolation	A router may help one request while harming system-wide fairness or budget allocation
Online adaptation	Benchmarks are largely offline and static compared with real traffic drift	Practical routers may need continual calibration, fallback logic, or exploration mechanisms
Cross-layer resource control	Router decisions are not deeply integrated with queueing, batching, GPU state, or cache locality	End-to-end efficiency depends on coordination between routing policy and lower-level resource management

6.6 Summary

The cross-paper synthesis suggests that LLM routing research is not converging on one canonical algorithm. Instead, it is branching into a family of designs that trade off simplicity, representational richness, and adaptivity against data collection cost and runtime

burden. This is a healthy sign for the field. It indicates that routing is becoming a mature systems-and-ML problem in which workload assumptions matter as much as model architecture.

Chapter 7

Conclusion and Future Work

This report analyzed three recent papers on LLM routing and argued that they collectively reveal a transition from query-level model selection to richer and more adaptive routing policies. RouteLLM demonstrates that preference-supervised binary routing can recover a large fraction of premium-model quality at much lower cost in a stable strong-weak setting. GraphRouter shows that routing benefits from explicit relational modeling of tasks, queries, and LLMs and that inductive model descriptors improve generalization to newly introduced models. Router-R1 goes further by allowing routing to unfold as a sequence of actions in which the policy LLM can reason, call models, observe intermediate information, and revise its decisions.

The main comparative conclusion is that recent routing research has evolved along three connected axes: routing granularity, context representation, and decision adaptivity. As one moves from RouteLLM to GraphRouter to Router-R1, the router is allowed to observe richer state, choose from a broader decision space, and optimize more complex objectives. However, each gain introduces new burdens in data dependence, training difficulty, runtime overhead, and evaluation complexity. The literature therefore does not support a simple claim that the most adaptive router is always best. Instead, it suggests that routing methods should be matched to workload structure, model-pool diversity, and deployment constraints.

For the purpose of this thesis-style report, the most important conclusion is that LLM routing has become a meaningful literature in its own right rather than a minor engineering heuristic. Future work will likely combine the low overhead of binary escalation, the richer representations of contextual multi-model routing, and the adaptive control of multi-round policies. The challenge is to integrate these ideas into routing systems that are not only accurate, but also efficient, robust, interpretable, and manageable under real deployment constraints.

Appendix A

Supplementary Notes on Benchmarks and Model Pools

A.1 Additional Notes on Benchmark Source Papers

Because this report is a literature review rather than an implementation paper, it is useful to restate what the benchmark source papers contribute conceptually. Chatbot Arena and MT-Bench are fundamentally about comparative or judged quality rather than strict one-answer correctness, which explains why they are particularly aligned with RouteLLM’s preference-learning story. MMLU and GSM8K, by contrast, make query difficulty look more like academic knowledge and explicit reasoning, which is why RouteLLM needs better-aligned augmentation to perform strongly there.

GraphRouter’s benchmark mix has a different function. Alpaca contributes instruction-following variety, SQuAD contributes grounded extraction, GSM8K contributes arithmetic and chain reasoning, and Multi-News contributes longer-form summarization behavior. That mixture is what makes GraphRouter’s contextual claim more interesting than a claim about one dataset family. Router-R1’s benchmark suite goes further still by isolating multi-hop and compositional QA as routing-relevant regimes. In that sense, the benchmark source papers are not background details. They partly determine what each routing paper is actually about.

Table A.1: Supplementary notes on benchmark source papers and their routing relevance.

Dataset / benchmark	Original paper emphasis	Why it matters to LLM routing
Chatbot Arena	Open human-preference platform for pairwise comparison of LLM responses	Makes comparative routing supervision possible without requiring one gold answer per query; especially suitable for binary strong-weak routing
MT-Bench	Multi-turn judged benchmark for assistant quality	Provides a more structured open-ended evaluation regime than raw arena logs, and is useful for measuring whether cheaper routing preserves judged quality
MMLU	Broad academic knowledge benchmark covering many subjects	Tests whether routers can distinguish between easy and difficult knowledge-heavy questions outside conversational preference data
GSM8K	Grade-school math word problems with explicit reasoning requirements	Reveals whether a router can detect when stronger reasoning models are needed rather than always relying on cheap default inference
Alpaca	Instruction-following dataset derived from self-instruct style generation	Useful for GraphRouter because it expands the task mix beyond canonical QA and exposes differences in instruction-following behavior across models
SQuAD	Reading-comprehension benchmark grounded in Wikipedia passages	Contributes a passage-grounded regime where model choice may depend on extraction and comprehension quality rather than free-form generation alone
Multi-News	Multi-document summarization benchmark	Introduces longer outputs and synthesis behavior, helping GraphRouter claim multi-task coverage beyond short-answer QA

Dataset / benchmark	Original paper emphasis	Why it matters to LLM routing
Natural Questions	Real search-style questions linked to Wikipedia evidence	Serves as an in-domain training and evaluation source for Router-R1, especially for general QA routing behavior
TriviaQA	Large distant-supervision QA benchmark	Useful for out-of-domain transfer because it differs in source and evidence profile from Natural Questions
PopQA	Entity-centric factual QA benchmark	Tests factual robustness and the limits of relying only on parametric memory in routing and retrieval settings
HotpotQA	Multi-hop QA with supporting-fact structure	A natural target for sequential routing, because relevant evidence often spans several steps
2WikiMultiHopQA	Multi-hop QA emphasizing explicit reasoning chains	Helps evaluate whether iterative model calls improve evidence chaining rather than only one-shot answer selection
MuSiQue	Compositional multi-hop questions created from simpler components	Particularly useful for testing decomposition-oriented routers such as Router-R1
Bamboogle	Benchmark for compositionality gaps in language models	Stress-tests whether routing and aggregation help when naive memorization is insufficient

A.2 Supplementary Notes on Candidate Model Pools

The three focal papers also differ sharply in the kind of model ecosystem they assume. RouteLLM assumes that the most important distinction is between a weak and a strong model, and its central experimental pair is GPT-4 versus Mixtral-8x7B. GraphRouter assumes a broader marketplace with ten candidate LLMs that differ in size and price, and therefore treats routing as a recommendation problem across a nontrivial pool. Router-R1 assumes yet another structure: a relatively small base policy model learns to coordinate a diverse routing pool of stronger external LLMs over multiple rounds.

These differences are analytically important because they shape what generalization means in each paper. In RouteLLM, generalization means adapting to new strong-weak pairs. In GraphRouter, it means incorporating unseen LLMs into a graph-based model-selection problem. In Router-R1, it means reasoning about descriptor-defined candidates during a sequential interaction process. The model pool is therefore not a background implementation detail; it is part of the problem definition.

Table A.2: Supplementary comparison of model-pool assumptions across the three focal papers.

Paper	Pool structure	Representative models mentioned in the paper	What this assumption enables or constrains
RouteLLM	Binary strong-weak pair	GPT-4 and Mixtral-8x7B in the main experiments; additional Claude and Llama-family pairs for transfer tests	Makes routing lightweight and interpretable, but constrains the action space to escalation rather than open multi-model choice
GraphRouter	Many-model one-shot pool	Ten LLMs of different sizes and costs, with Together API pricing used in the experimental setup	Enables explicit multi-model recommendation and new-LLM evaluation, but still assumes a one-shot decision after observing the query and graph context
Router-R1	Base policy model plus external routing pool	Two 3B-scale base policy models paired with a six-model routing pool spanning Qwen, Llama, Mistral, Mixtral, and Gemma families	Enables sequential coordination and aggregation, but introduces the highest control and runtime complexity of the three papers

A.3 Closing Note on Supplementary Material

The supplementary comparisons in this appendix reinforce a central claim of the thesis. Differences in routing results are not explained only by smarter algorithms. They are also explained by differences in supervision source, benchmark family, and the structure

of the candidate model pool. A mature literature review must therefore track all three dimensions together.

Bibliography

- [1] I. Ong, A. Almahairi, V. Wu, W. Chiang, J. E. Gonzalez, M. W. Kadous, I. Stoica, and T. Wu, “RouteLLM: Learning to route LLMs with preference data,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [2] T. Feng, Y. Shen, and J. You, “GraphRouter: A graph-based router for LLM selections,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [3] H. Zhang, T. Feng, and J. You, “Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning,” *arXiv preprint arXiv:2506.09033*, 2025.
- [4] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [5] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid LLM: Cost-efficient and quality-aware query routing,” *arXiv preprint arXiv:2404.14618*, 2024.
- [6] S. Chen, W. Jiang, B. Lin, J. Kwok, and Y. Zhang, “RouterDC: Query-based router by dual contrastive learning for assembling large language models,” *Advances in Neural Information Processing Systems*, 2024.
- [7] X. Dai, J. Li, X. Liu, A. Yu, and J. Lui, “Cost-effective online multi-LLM selection with versatile reward models,” *arXiv preprint arXiv:2405.16587*, 2024.
- [8] M. Šakota, M. Peyrard, and R. West, “Fly-swat or cannon? cost-effective language model choice via meta-modeling,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 606–615.
- [9] D. Stripelis, Z. Hu, J. Zhang, Z. Xu, A. D. Shah, H. Jin, Y. Yao, S. Avestimehr, and C. He, “TensorOpera Router: A multi-model router for efficient LLM inference,” *arXiv preprint arXiv:2408.12320*, 2024.
- [10] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “RouterBench: A benchmark for multi-LLM routing system,” *arXiv preprint arXiv:2403.12031*, 2024.

- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [12] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [13] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, “Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning,” *arXiv preprint arXiv:2503.09516*, 2025.
- [14] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, *et al.*, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [15] W. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. I. Jordan, J. E. Gonzalez, and I. Stoica, “Chatbot arena: An open platform for evaluating LLMs by human preference,” *arXiv preprint arXiv:2403.04132*, 2024.
- [16] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” in *International Conference on Learning Representations*, 2020.
- [17] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and chatbot arena,” in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [18] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [19] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following LLaMA model,” *Technical Report*, 2023.
- [20] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” *arXiv preprint arXiv:1606.05250*, 2016.
- [21] A. R. Fabbri, I. Li, T. She, S. Li, and D. R. Radev, “Multi-News: A large-scale multi-document summarization dataset and abstractive hierarchical model,” *arXiv preprint arXiv:1906.01749*, 2019.

- [22] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, *et al.*, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019.
- [23] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017, pp. 1601–1611.
- [24] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 9802–9822.
- [25] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” *arXiv preprint arXiv:1809.09600*, 2018.
- [26] X. Ho, A. D. Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 6609–6625.
- [27] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop questions via single-hop question composition,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.
- [28] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 5687–5711.
- [29] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [30] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017.
- [31] W. L. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [32] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.

- [33] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, “Self-instruct: Aligning language models with self-generated instructions,” *arXiv preprint arXiv:2212.10560*, 2022.
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [35] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 53 728–53 741, 2023.
- [36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.